IMPROVING LOGICAL REASONING OF LARGE LANGUAGE MODELS ON ARC VIA PROMPT ENGINEERING

Qining He
Fudan University
23307130016@m.fudan.edu.cn

ABSTRACT

Large language models (LLMs) have shown strong reasoning capabilities in many domains, yet their performance on abstract reasoning benchmarks such as the Abstraction and Reasoning Corpus (ARC) remains limited. In this work, we investigate whether prompt engineering alone can improve LLM performance on ARC-style grid transformation tasks, without modifying model parameters or introducing external supervision.

We evaluate five representative prompting strategies—baseline direct prediction, Chain-of-Thought (CoT), Reflexion, Program-Aided Language (PAL), and Self-Consistency—under a unified inference pipeline using the same non-deliberative LLM. Performance is measured using exact match accuracy.

Our results show that structured prompting substantially outperforms the baseline on standard ARC tasks, with CoT and PAL achieving the strongest gains. Through case studies, we find that tasks governed by a single dominant transformation rule and local spatial reasoning are consistently solvable across strategies, whereas tasks requiring compositional abstraction or global relational reasoning remain challenging.

These findings suggest that prompt engineering can improve robustness and reliability on moderately complex ARC tasks, but remains insufficient for inducing deeper abstract reasoning.

1 INTRODUCTION

Large language models (LLMs) have achieved impressive results across a wide range of language and reasoning tasks. However, their ability to perform abstract reasoning—particularly in settings that require strong inductive biases and minimal supervision—remains limited. The Abstraction and Reasoning Corpus (ARC) (`https://arcprize.org/arc-agi`) is a representative benchmark in this regard, requiring models to infer latent transformation rules from only a few examples and apply them to novel inputs.

ARC tasks are intentionally designed to resist surface-level pattern matching. Each task consists of a small set of input–output grid pairs that implicitly define an abstract rule, such as object-level transformations, symmetry, or spatial relations. While humans can often solve these tasks by leveraging compositional reasoning, current LLMs struggle to reliably acquire such abstractions from textual descriptions alone.

Recent studies have shown that prompt engineering can significantly influence the reasoning behavior of LLMs without parameter updates. Techniques such as Chain-of-Thought prompting, program-aided reasoning, self-consistency, and reflective prompting have led to notable improvements on mathematical and symbolic reasoning benchmarks. However, their effectiveness on ARC—where reasoning is highly structured, visual, and rule-based—remains insufficiently understood.

In this work, we conduct a systematic empirical study of prompting strategies for ARC task solving. We compare a simple baseline prompt with four structured approaches: Chain-of-Thought, Reflexion, Program-Aided Language, and Self-Consistency. Rather than aiming for state-of-the-art performance, our goal is to analyze how different prompt designs affect accuracy, robustness, and failure modes in abstract reasoning tasks.

Through quantitative evaluation and qualitative case analysis, we show that structured prompting can substantially improve performance on standard ARC tasks, but also exposes clear limitations. While certain strategies better exploit local regularities and dominant rules, none reliably solve tasks requiring compositional or relational abstraction. These findings clarify both the strengths and the boundaries of prompt engineering for abstract reasoning.

2 METHODS

In this work, we systematically investigate how different prompt engineering strategies influence large language models' ability to perform abstract reasoning on ARC tasks.

All strategies are evaluated under the same pipeline, using identical data, model settings, and output parsing procedures. They differ only in how reasoning is elicited, structured, or constrained through prompt design.

Below, we describe each prompting strategy, including its motivation, design choices, and role in our experimental comparison.

2.1 BASELINE PROMPT

The baseline prompt serves as a minimal reference for ARC reasoning without explicit guidance. It provides the model with a set of training input–output grid pairs and a test input grid, and directly instructs the model to infer the transformation rule and return the output grid.

No intermediate reasoning steps, explanations, or verification mechanisms are required. The model is constrained to output only the final grid in a fixed JSON-style format. This setup evaluates the model's ability to solve ARC tasks relying solely on its implicit pattern recognition and internal representations, and establishes a lower bound for comparison with more structured prompting strategies.

2.2 CHAIN-OF-THOUGHT PROMPT

The Chain-of-Thought (CoT) prompt (Wei et al., 2023) explicitly encourages the model to externalize its reasoning process. In addition to the training examples and test input, the model is instructed to analyze each training example step by step, summarize the inferred transformation rule, and explain how the rule applies to the test grid before producing the final output.

The response is structured into a reasoning section followed by a final answer section. This design aims to improve abstract rule induction by decomposing ARC tasks into explicit reasoning steps, while also increasing interpretability and enabling qualitative error analysis.

2.3 REFLEXION PROMPT

The Reflexion prompt (Shinn et al., 2023) introduces a verification-aware reasoning loop to mitigate a common ARC failure mode: proposing a plausible rule that does not generalize across all training examples. Reasoning is decomposed into two roles: an *Actor*, which proposes a transformation rule and predicts the test output, and an *Evaluator*, which checks whether the proposed rule correctly reproduces all training outputs.

If the proposed rule fails verification, the Actor is allowed a single revision step to refine the rule and regenerate the output. This closed-loop design explicitly enforces hypothesis checking against observed evidence, while limiting the revision depth to control computational cost and avoid divergence.

2.4 PROGRAM-AIDED LANGUAGE PROMPT

The Program-Aided Language (PAL) prompt (Gao et al., 2023) requires the model to translate its inferred transformation rule into an executable Python function. After reasoning about the task in natural language, the model must output a function `transform(grid)` that implements the transformation algorithmically.

Table 1: Summary of prompt strategies.

Strategy	Core mechanism
Baseline	Direct prediction
CoT	Step-by-step reasoning
Reflexion	Verification and revision
PAL	Algorithmic reasoning
Self-Consistency	Consistency-based selection

To ensure stability and safety, the generated code is restricted to basic Python constructs such as loops, conditionals, and indexing, with no imports, recursion, or side effects. The program is executed externally to produce the final output grid. By enforcing executable structure, PAL aims to reduce ambiguity and execution errors in spatially precise ARC tasks.

2.5 SELF-CONSISTENCY PROMPT

The self-consistency prompt (Wang et al., 2023) addresses the brittleness of single-pass reasoning by encouraging hypothesis diversity within a single model response. The model is instructed to generate multiple independent solution attempts (three in our experiments), each consisting of a brief rule description and a corresponding output grid for the test input.

After generation, the final prediction is selected via majority voting over the output grids. The output that appears most frequently among the attempts is chosen, with a deterministic fallback in case of ties. This prompt-level self-consistency variant approximates ensemble reasoning without requiring multiple model calls, and isolates the effect of aggregating independent reasoning paths on ARC performance.

2.6 SUMMARY OF PROMPTING STRATEGIES

Table. 1 summarizes the core mechanisms of all prompting strategies evaluated in this work.

3 EXPERIMENTS

In this section, we describe the datasets, model configuration, evaluation protocol, and experimental setup used to systematically compare different prompt engineering strategies on ARC tasks.

3.1 DATASETS

We evaluate our prompting strategies on a subset of tasks from the Abstraction and Reasoning Challenge (ARC).

Each ARC task consists of multiple training input–output grid pairs and one test input grid, where the goal is to infer the underlying transformation rule from the training examples and apply it to the test input.

To better analyze model behavior across different difficulty levels, we construct two evaluation subsets:

- ARC-Standard: A mixed-difficulty subset containing tasks ranging from relatively simple pattern transformations to more complex abstract reasoning problems, which contains 30 tasks.
- ARC-Hard: A challenging subset consisting exclusively of difficult tasks, designed to stress-test the model's abstraction and generalization abilities. This subset contains 141 tasks in total.

Table 2: Overall Results

Strategy	ARC-Standard Accuracy (%)	ARC-Hard Accuracy (%)
Baseline	33.33	2.13
CoT	60	6.38
Reflexion	30	2.13
PAL	60	10.64
Self-Consistency	36.67	3.55

3.2 Model and Inference Configuration

All experiments are conducted using DeepSeek-V3.2 (non-thinking mode), as required by the course evaluation protocol.

We use a unified inference configuration across all prompt strategies to ensure fair comparison:

- Model: DeepSeek-V3.2 (non-thinking mode)
- Temperature: 1.0
- Max tokens: 8,000
- Other parameters: Default values as specified in the official API

No additional fine-tuning or external tools are used. All improvements stem solely from differences in prompt design and reasoning structure.

3.3 Evaluation Metric

We evaluate model performance using Accuracy, defined as the proportion of tasks for which the model's predicted output grid exactly matches the ground truth.

Formally, given a predicted grid P and a ground truth grid G, a prediction is considered correct if and only if:

$$\forall(i, j), P_{i,j} = G_{i,j}$$

If this condition holds, the prediction receives a score of 1; otherwise, it receives a score of 0. The final accuracy is computed as the average score across all evaluated tasks.

This Exact Match criterion provides a rigorous measure of abstract reasoning correctness.

3.4 Overall Results

Table. 2 summarizes the overall accuracy of different prompting strategies on both evaluation subsets.

4 Results and Analysis

4.1 Performance Comparison on Standard ARC Tasks

We evaluate five prompting strategies on 30 standard ARC tasks. The overall accuracy results are summarized as follows:

- Baseline: 33.33% (10/30)
- Chain-of-Thought (CoT): 60.00% (18/30)
- Reflexion: 30.00% (9/30)
- Program-Aided Language (PAL): 60.00% (18/30)
- Self-Consistency: 36.67% (11/30)

Among all strategies, CoT and PAL achieve the highest accuracy, both correctly solving 18 out of 30 tasks. In contrast, the baseline and Reflexion strategies perform substantially worse, with Reflexion even underperforming the baseline.

4.2 Strengths and Weaknesses of Different Strategies

Chain-of-Thought (CoT) demonstrates strong effectiveness on tasks requiring multi-step reasoning and explicit abstraction of transformation rules. By externalizing intermediate reasoning steps, CoT appears to reduce reasoning errors caused by implicit pattern inference, leading to a significant improvement over the baseline.

Program-Aided Language (PAL) matches CoT in overall accuracy, suggesting that explicitly representing transformations as executable programs can be equally effective for ARC tasks. PAL performs particularly well on tasks with clear algorithmic structure, such as repetitive spatial transformations or rule-based color mappings.

In contrast, **Reflexion** fails to improve performance in our experiments and even underperforms the baseline. We attribute this degradation primarily to the difficulty of reliably verifying abstract transformation rules in the ARC setting. Applying a hypothesized rule to all training grids requires precise spatial execution, which remains challenging for language models. As a result, the evaluator may incorrectly judge a correct but imperfectly applied rule as invalid, prematurely rejecting otherwise valid abstractions. Moreover, the quality of the feedback generated by the evaluator is not consistently reliable, which can misguide the subsequent revision step and amplify errors rather than correct them. These factors together limit the effectiveness of verification-driven reasoning for ARC tasks.

Self-Consistency yields only a modest improvement over the baseline. Unlike standard self-consistency approaches that rely on multiple independent stochastic samples, our cost-constrained variant requires the model to generate several solution attempts within a single response. This design is subject to a fixed maximum token budget, which reduces the depth and completeness of each individual attempt. Consequently, the generated hypotheses tend to be shallower and less diverse, limiting the effectiveness of majority voting. Under these constraints, self-consistency mitigates random variability but does not fundamentally resolve errors arising from incorrect rule induction, which remain the dominant failure mode in ARC tasks.

Overall, strategies that explicitly structure reasoning or computation (**CoT** and **PAL**) outperform those relying on post-hoc correction or majority voting.

4.3 Case Study: Universally Solved Tasks

To better understand the characteristics of ARC tasks that are consistently solvable, we analyze tasks that are successfully solved by all prompting strategies. Two representative examples are shown in Fig. 1.

In Task 1, the rule consists of a local geometric operation: each blue cell is reflected across its uniquely adjacent gray anchor cell, with no interaction between different object pairs. In Task 2, the transformation reduces to a deterministic spatial rescaling: colored cells within a fixed 5×5 region are compressed into a 3×3 output grid while preserving their relative positions.

Second, these tasks rely predominantly on local or low-order spatial relationships. The correct output can be determined by inspecting small neighborhoods (Task 1) or by applying a uniform global mapping that does not require reasoning about long-range dependencies or object interactions (Task 2).

Finally, object boundaries and color semantics are unambiguous. Colors serve clear functional roles (e.g., anchor versus movable object in Task 1), and there is little risk of confusing foreground and background elements.

As a result, even the baseline prompt—without explicit reasoning guidance—is often sufficient to recover the correct transformation. More structured prompting strategies such as Chain-of-Thought and Program-Aided Language primarily improve robustness and consistency rather than enabling qualitatively new forms of abstraction. This suggests that these tasks mainly assess reliable pat-

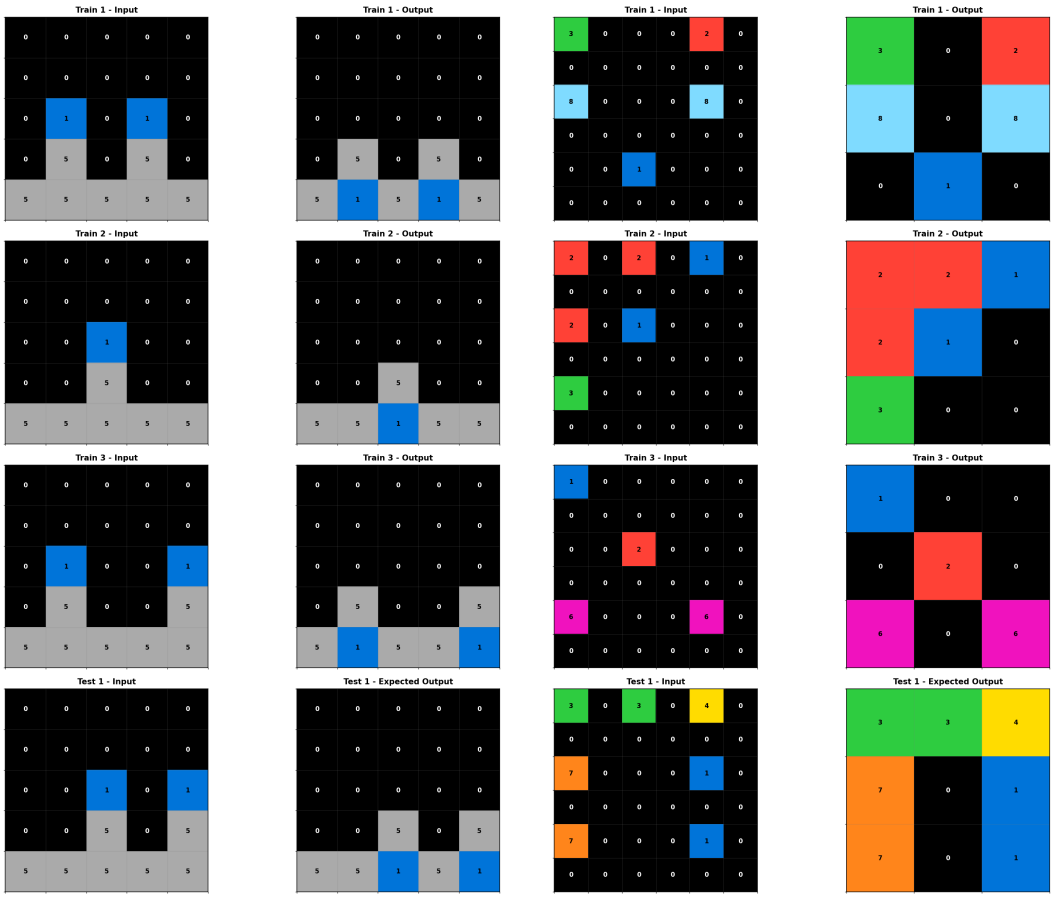


Figure 1: Two representative ARC tasks that are consistently solved by all prompting strategies.

tern recognition and straightforward spatial mapping, rather than deep compositional or relational reasoning.

4.4 Case Study: Universally Failed Tasks

We also analyze tasks that none of the strategies successfully solve. Two representative examples are shown in Fig. 2

these universally failed tasks exhibit several common characteristics that fundamentally challenge prompt-based reasoning.

First, the underlying transformation rules are compositional and multi-stage. In the first task, the grid must be decomposed into multiple spatially separated subregions (the four corner blocks), each of which requires identifying a missing color by reasoning over set complements within a local color inventory, followed by a selective replacement operation. The correct transformation cannot be described as a single local mapping, but instead requires coordinating multiple independent inferences under a shared global structure.

Second, these tasks impose strong global structural constraints. In the second task, the model must infer the existence of an implicit geometric object—a partially specified 3×3 square—whose completion depends on global shape reasoning, obstacle avoidance, and role-specific color assignments (boundary completion versus interior filling). Solving the task requires jointly reasoning about geometry, spatial continuity, and exclusion constraints imposed by immutable obstacle cells.

Third, the required reasoning is implicitly relational rather than perceptual. Critical information is not encoded in visually salient patterns alone, but in relationships such as “missing element,”

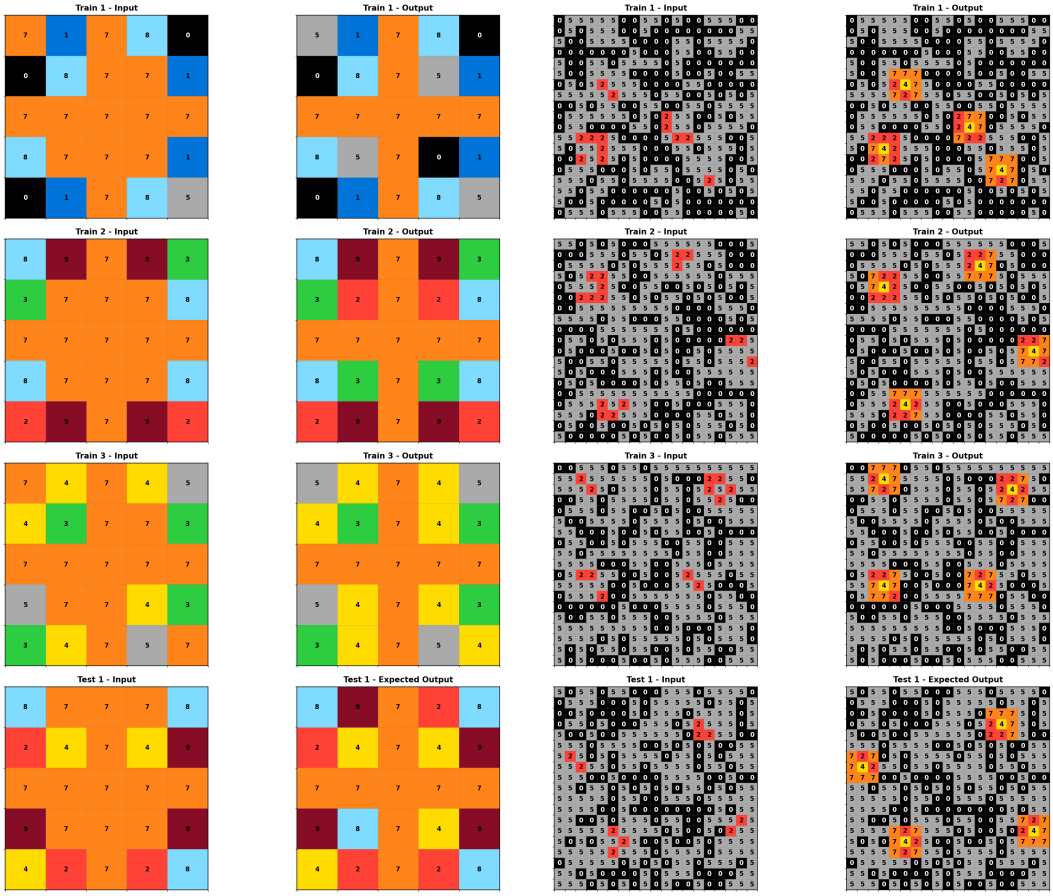


Figure 2: Two representative ARC tasks that aren't solved by any prompting strategy.

"enclosed region," or "completion under constraints," which are not directly observable from local neighborhoods.

Across all prompting strategies, failure modes are strikingly consistent: models often identify plausible local patterns but fail to integrate them into a coherent global hypothesis, leading to incomplete, overgeneralized, or structurally invalid outputs. Notably, explicit reasoning (Chain-of-Thought), executable programs (PAL), and sampling-based aggregation (Self-Consistency) do not substantially alleviate these failures, suggesting that the bottleneck lies in abstract relational inference rather than reasoning format or prompt structure.

Overall, these cases highlight a fundamental limitation of current prompt-based approaches: while they can enhance reliability on tasks governed by simple, localized rules, they remain inadequate for ARC problems that require compositional abstraction, global constraint satisfaction, and implicit relational reasoning.

5 Discussion and Future Work

5.1 Discussion

This work provides an empirical investigation into the relationship between prompting strategies and abstract reasoning in the context of ARC tasks. Our results suggest that while carefully designed prompting strategies can substantially improve model robustness and reliability, their ability to induce genuinely abstract reasoning remains limited.

Across multiple prompting strategies, including Chain-of-Thought (CoT), Program-Aided Language models (PAL), and self-reflective variants, performance gains are primarily observed in tasks with clear and unambiguous transformation rules. As illustrated by the case studies, such tasks often admit a single dominant pattern that can be inferred from local visual cues, without requiring hierarchical abstraction or long-range spatial reasoning. In these settings, structured prompting mainly serves as a regularization mechanism, reducing brittle failures rather than enabling qualitatively new forms of reasoning.

Conversely, tasks that require deeper abstraction, global relational understanding, or flexible concept composition remain challenging across all strategies. This observation highlights a fundamental limitation of prompt-based approaches: prompting can guide how a model reasons, but cannot fully compensate for missing internal representations or inductive biases. As a result, improvements from prompting should be interpreted as better utilization of existing capabilities, rather than the emergence of new reasoning primitives.

Overall, our findings suggest that prompt engineering functions as an interface-level optimization between the task and the model, rather than a substitute for architectural or representational advances. In the context of abstract reasoning benchmarks such as ARC, this distinction is particularly important when interpreting performance gains.

5.2 Future Work

The observations in this study naturally point to several promising directions for future research.

First, integrating prompting strategies with architectures explicitly designed for abstraction may offer a more effective path forward. Hybrid systems that combine neural models with symbolic, programmatic, or object-centric representations could better capture the compositional structure underlying ARC tasks, while still benefiting from the flexibility of natural language prompting.

Second, future work could explore adaptive or learned prompting mechanisms, where prompts are dynamically generated or refined based on task feedback. Such approaches may help bridge the gap between static prompt templates and task-specific reasoning requirements, particularly in settings with heterogeneous abstraction levels.

Third, extending the evaluation beyond final accuracy to include intermediate reasoning signals represents another important direction. Analyzing the consistency, stability, and faithfulness of intermediate reasoning traces may provide deeper insights into when prompting genuinely supports reasoning, as opposed to merely shaping outputs.

Finally, from a prompt design perspective, future ARC-oriented work may benefit from incorporating richer perceptual grounding and contextual guidance. One promising direction is to combine existing strategies with *vision-language prompting* (Zhou et al., 2022), where explicit textual descriptions of visual structures (e.g., objects, symmetry, or spatial relations) are introduced to guide abstraction. Another complementary avenue is the use of carefully curated *few-shot in-context learning (ICL)* (Li et al., 2023), where example tasks exhibiting similar transformation patterns are provided to bias the model toward appropriate hypothesis spaces. Exploring how such perceptually grounded or context-enhanced prompts interact with reasoning-oriented strategies like CoT, PAL, or self-consistency may offer a practical path toward more robust abstraction in ARC tasks.

References

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models, 2023. URL https://arxiv.org/abs/2211.10435.

Tianle Li, Xueguang Ma, Alex Zhuang, Yu Gu, Yu Su, and Wenhu Chen. Few-shot in-context learning for knowledge base question answering, 2023. URL https://arxiv.org/abs/2305.01750.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023. URL https://arxiv.org/abs/2203.11171.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL https://arxiv.org/abs/2201.11903.

Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Learning to prompt for vision-language models. *International Journal of Computer Vision*, 130(9):2337–2348, July 2022. ISSN 1573-1405. doi: 10.1007/s11263-022-01653-1. URL http://dx.doi.org/10.1007/s11263-022-01653-1.